

Mission Sentinel-X

Rapport d'ingénierie technique - consortium VIG1L-8 (AetherCorp)

Groupe	G04
Nœud	VIG1L-8-NODE04 (ESP32)
Option matérielle	B - Topologie distribuée « Edge-to-Server » : PC Serveur Local = ordinateur portable Linux d'un membre du groupe
Dépôts	github.com/ESPI-Workshop-Vig1l-8 : infra , backend , dashboard , firmware , IA_Vision , IA_Predictions

Équipe

Membre	Filière	Rôle principal
Julien DAUVERGNE	INFRA / CYBER	Infrastructure, réseau, sécurité, firmware ESP32, intégration
Lucas DIOT	WEB	Backend et dashboard
Rem TSHIMANGA	IA	Vision (YOLO) et maintenance prédictive
Latif MOUINOU	CYBER	Sécurité

Sommaire

- | | |
|-------------------------------------|--|
| 1. Synthèse | 7. Backend, API et dashboard |
| 2. Contexte et choix d'architecture | 8. Matrice de sécurité (TLS, durcissement) |
| 3. Réseau de table | 9. Documentation de l'IA |
| 4. Électronique et firmware | 10. Tests et validation |
| 5. Infrastructure conteneurisée | 11. Limites et améliorations |
| 6. Séquences des échanges réseau | 12. Annexes |

Conventions

FAIT réalisé et testé. Les noms techniques (topics, champs JSON, variables) sont en `police fixe`.

1. Synthèse

Sentinel-X est un boîtier de surveillance autonome pour les micro-centrales d'AetherCorp. Un nœud ESP32 mesure la température, l'humidité, les gaz combustibles et la présence, et transmet ses mesures en **MQTT chiffré (TLS)** au **PC Serveur Local**. Celui-ci exécute, dans des conteneurs Docker isolés, le broker MQTT, la base de données, l'API, le tableau de bord et deux intelligences artificielles : une **détection d'intrusion par vision** (webcam USB, YOLOv8n) et une **maintenance prédictive** (Isolation Forest) qui repère les dérives avant tout seuil critique.

TLS 1.2+
de l'ESP32 au broker, CA locale vérifiée par le nœud

2 niveaux
d'alerte IA : avertissement puis confirmation (LED du nœud)

10 / 10
dérives lentes détectées (données simulées), 0,07 % de fausses alertes

37 ms
par image pour la vision (PC de développement, CPU) - objectif < 100 ms

Réalisation assistée par IA (Claude Code) pour la gestion git, le code du frontend et du backend, et les tests du backend Go et de l'API.

Ce qui est livré

Brique	Contenu	État
Nœud ESP32	Firmware C++ (PlatformIO) : capteurs, JSON, MQTTS, LED d'alerte, diagnostic matériel	FAIT
Infrastructure	Docker Compose : Mosquitto, CouchDB, backend Go, dashboard, IA vision, IA prédiction ; scripts de certificats et de comptes	FAIT
Backend / dashboard	Ingestion validée, API REST + WebSocket, historique, alertes, commandes, export CSV d'entraînement	FAIT
IA	Vision MOG2 + YOLOv8n ; Isolation Forest entraîné sur les vraies mesures, détection en temps réel	FAIT
Sécurité	TLS, comptes et ACL par nœud, rôles API, conteneurs non root, secrets hors dépôt	FAIT
Exposition de l'hôte	MQTTS publié sur l'adresse du hotspot uniquement, dashboard sur la boucle locale, base non publiée (section 8.3)	FAIT
Mesures réelles	Réseau ESP32 ↔ serveur 1 à 2 ms, pile complète ≈ 790 Mio de RAM, modèle entraîné sur 1 h de mesures en salle	FAIT

2. Contexte et choix d'architecture

2.1 Le besoin

Le sujet impose quatre piliers : **Edge Computing & IoT** (un microcontrôleur unique qui capte et alerte localement), **IA locale** (vision sur webcam USB et détection prédictive sur séries temporelles, sans seuils statiques), **infrastructure** (conteneurs résilients, réseau de table isolé) et **cybersécurité** (chiffrement de bout en bout, durcissement). L'interconnexion de toutes les briques est éliminatoire pour la démonstration.

2.2 Option matérielle

Le groupe a retenu l'**option B**, « **Edge-to-Server** » : le Raspberry Pi est retiré et le rôle de PC Serveur Local est tenu par l'**ordinateur portable Linux** d'un membre du groupe. Il exécute toute la pile Docker Compose, porte la webcam USB et sert de point d'accès Wi-Fi (hotspot) au réseau de table. Le boîtier Sentinel-X ne contient que l'ESP32 et ses capteurs.

- **Pourquoi** : la pile a été développée et validée sur ce PC ; l'IA profite de son processeur (vision à 37 ms par image) ; aucun écart entre l'environnement de développement et celui de la démonstration.
- **Conséquence réseau** : la carte Wi-Fi du portable est utilisée par le hotspot et ne peut pas être connectée à un autre réseau en même temps. Les images sont donc construites avec Internet *avant* la démonstration, puis la pile fonctionne hors ligne.
- **Conséquence sécurité** : le serveur est un poste personnel. La sécurité porte sur ce qui est exposé au réseau de table : un seul port, lié à l'adresse du hotspot (section 8.3).
- **Portabilité** : les images restent compatibles `linux/arm64` ; un passage à l'option A (Raspberry Pi 5) resterait possible sans modifier la pile.

2.3 Écarts au matériel prévu

Prévu par le sujet	Utilisé	Raison et conséquences
ESP8266	ESP32 (DevKit)	Seul microcontrôleur disponible. Avantages : plus de broches, ADC 12 bits (ADC1 utilisable avec le Wi-Fi), pile TLS plus confortable (mbedtls).
Buzzer piézoélectrique	2 LED (mouvement, environnement)	Plus de buzzers disponibles. La LED environnement porte les alertes : fixe = premier avertissement local, clignotante = alerte confirmée par l'IA.
Écran OLED I2C	-	Plus d'écrans disponibles. L'état du nœud (IP, connexion, mesures) est visible sur le dashboard et le moniteur série.

2.4 Principes de conception

- **Une seule source de vérité** : toutes les données (mesures, événements, alertes, annotations) sont dans CouchDB.
- **Un seul écrivain** : seul le backend écrit en base, après validation. L'IA lit en lecture seule des données sûres.
- **Validation à l'entrée** : chaque message MQTT est vérifié (format, cohérence identifiant/topic, plages de valeurs).

- **Moindre privilège** : un compte par nœud et par service, rôles distincts, conteneurs non root, ports minimaux.
- **Pas de secret dans le code** : mots de passe, jetons et clés vivent dans des fichiers ignorés par git.
- **Défaillances visibles** : un capteur en panne envoie `null`, une coupure de série est détectée ; rien n'est masqué.
- **Reproductible** : tout se déploie avec Docker Compose et deux scripts ; les images sont construites depuis les dépôts.

3. Réseau de table

3.1 Plan d'adressage

Élément	Valeur	Justification
Réseau	192.168.10.0/24 , SSID VIGIL8_HOTSPOT	Réseau dédié à la table, isolé des autres groupes
Sécurité Wi-Fi	WPA2-PSK, bande 2,4 GHz	L'ESP32 ne capte pas le 5 GHz
PC Serveur Local	192.168.10.1 - passerelle et DHCP	Adresse inscrite dans le certificat TLS du broker
Nœud ESP32	192.168.10.5 (IP fixe)	Hors de la plage DHCP : aucun conflit possible avec un autre client
Plage DHCP	192.168.10.10 à .254	Clients occasionnels (aucun service ne leur est destiné)

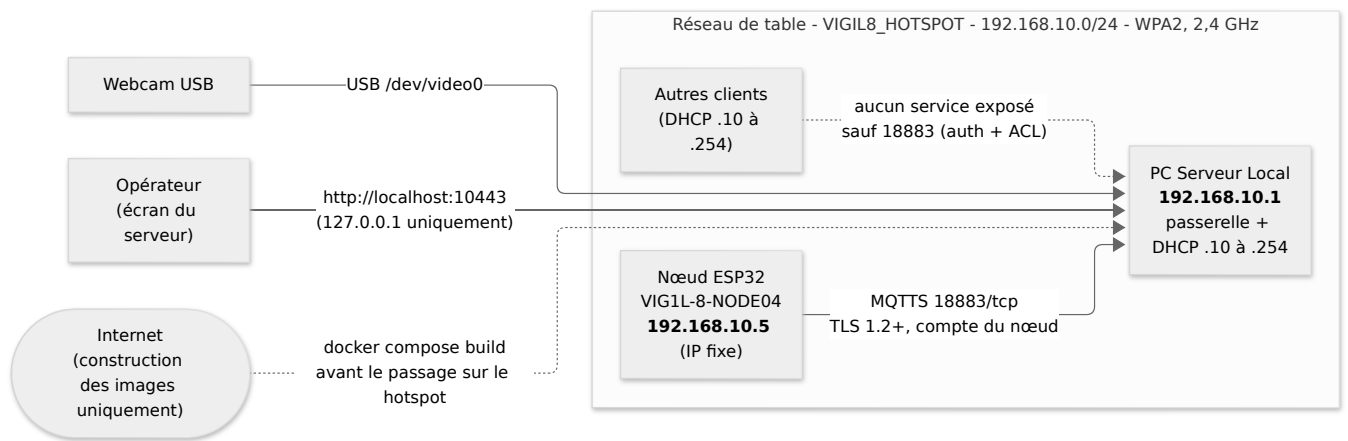


Figure 1 - Réseau de table : seul le port MQTTS 18883 est exposé au réseau ; le dashboard n'écoute que sur la boucle locale du serveur (option B : l'opérateur travaille sur le portable qui sert de serveur).

3.2 Matrice des flux

Source → destination	Port / protocole	Chiffrement	Authentification	Exposition
ESP32 → broker	18883/tcp (→ 8883) MQTT	TLS 1.2+, CA locale	compte du nœud + ACL	réseau de table
Navigateur → dashboard	10443/tcp HTTP	- (boucle locale)	clé d'accès (jeton API)	127.0.0.1 uniquement
backend ↔ broker	1883/tcp MQTT	réseau Docker interne	compte backend	non publié
backend → CouchDB	5984/tcp HTTP	réseau Docker interne	utilisateur backend (écriture)	non publié
ia-prediction → CouchDB	5984/tcp HTTP	réseau Docker interne	utilisateur ia (lecture seule)	non publié

IA → backend	5000/tcp HTTP	réseau Docker interne	jeton service	non publié
dashboard → ia-vision	8000/tcp MJPEG	réseau Docker interne	clé d'accès vérifiée par le backend	non publié

3.3 Isolation

- **Réseau de table** : hotspot dédié du serveur (WPA2) ; aucun autre groupe ne partage ce réseau.
- **Surface exposée** : un seul port sur le réseau (MQTT 1883), protégé par TLS, mot de passe et ACL. Sur l'hôte, seul le dashboard est publié, et uniquement sur 127.0.0.1 : la base n'est joignable que depuis les conteneurs du backend et de l'IA. L'export CSV d'entraînement passe par l'API (le backend lit la base et sert le fichier).
- **Dashboard sur la boucle locale** : avec l'option B, le serveur est le portable personnel de l'opérateur, qui consulte le dashboard sur cette même machine. Le publier sur le réseau ne servirait à rien et exposerait une interface d'administration (commandes, acquittements, flux caméra) aux autres clients du hotspot. Avec un serveur dédié (option A), il serait publié sur un port du réseau privé d'administration, en HTTPS avec la CA locale, et filtré par le pare-feu. Accès ponctuel depuis un autre poste : tunnel SSH (`ssh -L 10443:127.0.0.1:10443 <serveur>`).
- **Ports non standard** : MQTT sur 1883 (au lieu de 8883) et dashboard sur 10443 . Les attaques automatisées (robots, scripts de scan rapides, vers) visent les ports par défaut des services courants (1883/8883 pour MQTT, 80/443/8080 pour le web, 5984 pour CouchDB) : décaler les ports les évite et réduit le bruit dans les journaux. C'est une mesure complémentaire, pas une protection en soi : un scan complet (`nmap -p-`) les retrouve, la sécurité repose sur TLS, l'authentification et les ACL.
- **Réseaux Docker par paire de services** : chaque conteneur ne voit que les services dont il a besoin (section 5).
- **Port lié au hotspot** : 1883 est publié sur 192.168.10.1 uniquement, pas sur toutes les interfaces. Docker contourne le pare-feu de l'hôte pour les ports publiés : sans cette liaison, le broker serait joignable depuis le Wi-Fi de l'école ou l'Ethernet. Le hotspot doit donc être actif avant le démarrage de la pile.

4. Électronique et firmware

4.1 Composants

Composant	Mesure / rôle	Alimentation	Broche ESP32
DHT22	Température, humidité (1 lecture / 2 s max)	3,3 V par GPIO 33	GPIO 27
MQ-2	Gaz combustibles, fumées (sortie analogique AO)	5 V (chauffe ~150 mA)	GPIO 34 (ADC1), en direct
HC-SR501	Présence infrarouge (sortie 3,3 V)	5 V	GPIO 14
LED mouvement + résistance	Allumée pendant une détection du PIR	-	GPIO 25
LED environnement + résistance	Fixe : plafond local ; clignotante : alerte IA	-	GPIO 26
LED intégrée	Suit le PIR	-	GPIO 2

Sentinel-X - câblage du nœud VIG1L-8-NODE04 (ESP32)

Deux étiquettes de même nom sont reliées · GND commun à tous les composants · LED : patte longue (anode) côté résistance

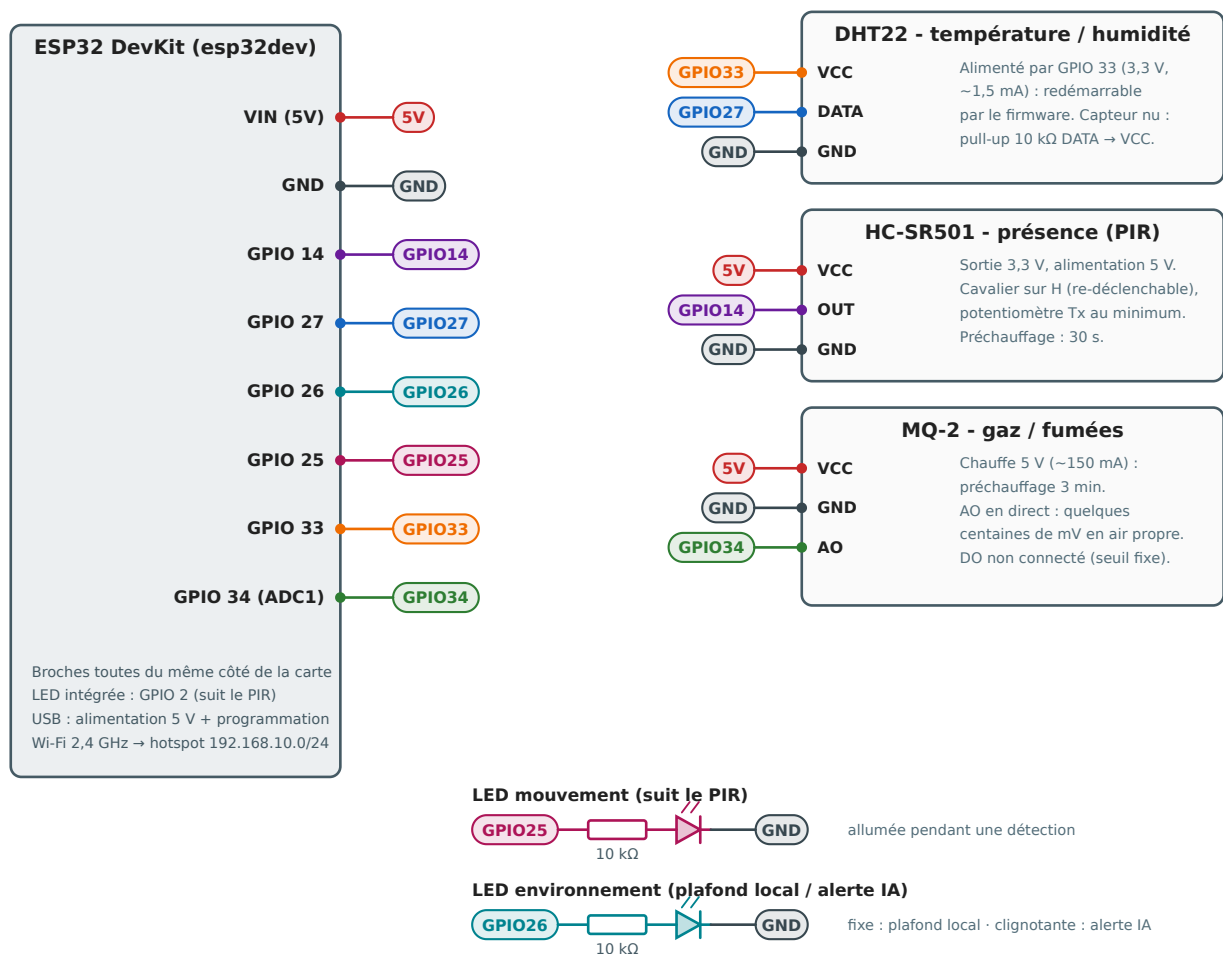


Figure 2 - Câblage du nœud VIG1L-8-NODE04 (source : `firmware/include/config.h`).

4.2 Choix électroniques

- **Une seule rangée de broches** : sur la breadboard, l'ESP32 ne laisse qu'une rangée libre, d'un seul côté. Toutes les broches utilisées sont donc du même côté de la carte (VIN, GND, 14, 27, 26, 25, 33, 34).
- **MQ-2 en 5 V, AO en direct** : le MQ-2 doit être alimenté en 5 V (sa résistance chauffante ne chauffe pas assez en 3,3 V). En air propre, sa sortie AO reste à quelques centaines de mV, bien dans la plage de l'ADC (0 à ~3,1 V) : elle est branchée directement sur GPIO 34 et le firmware publie la tension lue (facteur `MQ2_DIVIDER` = 1). Sous une forte concentration de gaz, AO peut dépasser 3,3 V (voir les limites, section 11).
- **ADC1 (GPIO 32 à 39)** : les broches de l'ADC2 ne sont plus lisibles quand le Wi-Fi est actif ; GPIO 34 est sur l'ADC1.
- **DHT22 alimenté par GPIO 33** : la broche 3V3 est de l'autre côté de la carte. Une sortie à l'état haut fournit 3,3 V et une vingtaine de mA, le capteur en consomme ~1,5 mA. Un pont de résistances ne conviendrait pas (sa tension chute dès que le capteur consomme), et le 5 V mettrait DATA à 5 V sur l'ESP32. Bonus : le firmware peut couper et rallumer le capteur.
- **LED** : une résistance de 10 kΩ en série sur chaque LED (seule valeur disponible), qui limite le courant dans la broche.
- **Broches évitées** : 0, 2 (sauf LED intégrée), 5, 12, 15 (démarrage), 6 à 11 (mémoire flash), 34 à 39 réservées aux entrées.
- **HC-SR501** : cavalier sur H (re-déclenchable), délai Tx au minimum (~3 s), 30 s de calibration au démarrage.

4.3 Firmware

C++ avec PlatformIO (framework Arduino-ESP32 2.0.17), bibliothèques PubSubClient (MQTT), ArduinoJson 7 et DHT (Adafruit). Deux environnements : `esp32dev` (firmware Sentinel-X) et `diagnostic` (test du câblage sans réseau).

- **Configuration** : `include/config.h` (versionné : identifiant, IP, broches, seuils) et `include/secrets.h` (ignoré par git : Wi-Fi, mot de passe MQTT, certificat de la CA). Sans `secrets.h`, la compilation s'arrête avec un message explicite.
- **Redémarrage du DHT22** : après 5 erreurs consécutives (10 s), le capteur est coupé 1 s puis rallumé ; les mesures restent publiées en erreur (`null`) pendant ce temps.
- **Boucle non bloquante** : reconnexion Wi-Fi et MQTT en arrière-plan ; les capteurs sont lus même pendant une coupure réseau.
- **Publication** : télémetrie toutes les 2 s, événement immédiat à chaque changement d'état du PIR, statut retenu et message de « dernière volonté » (LWT) `{"online":false}`.
- **Commandes** : `{"strobe":true,"duration_s":8}` fait clignoter la LED environnement (1 à 60 s).
- **Avertissement local** : LED environnement fixe au-dessus de 40 °C ou 800 mV (hystérésis 1 °C / 100 mV, gaz ignoré pendant le préchauffage). C'est un premier signal local : la décision d'anomalie reste à l'IA.

4.4 Format des données (v1)

Topic	Sens	Quand
<code>vigil8/<id>/telemetry</code>	nœud → serveur	toutes les 2 s
<code>vigil8/<id>/event</code>	nœud → serveur	changement d'état du PIR

vigil8/<id>/status	nœud → serveur (retenu)	connexion ; LWT publié par le broker si le nœud disparaît
vigil8/<id>/cmd	serveur → nœud	alerte confirmée ou commande opérateur

```
{ "v": 1, "device_id": "VIG1L-8-NODE04", "seq": 18342, "uptime_ms": 36684012,
  "temp_c": 25.8, "hum_pct": 61.5, "gas_mv": 259, "pir": false, "pir_events": 0,
  "status": { "dht": "ok", "gas_warm": true, "env_warn": false, "rssi": -58 } }
```

- temp_c / hum_pct valent null si le DHT22 ne répond pas (pas de « dernière valeur connue » qui masquerait une panne).
- gas_mv : tension de sortie du MQ-2 en mV, non étalonnée (pas de ppm sans étalonnage en air propre).
- seq : compteur par topic. Un trou signale un message perdu, un retour à 0 avec un petit uptime_ms un redémarrage.
- status.gas_warm : faux pendant les 3 minutes de préchauffage, pour exclure ces mesures de l'entraînement.

5. Infrastructure conteneurisée

5.1 Services

Service	Image / technologie	Rôle	Utilisateur	Exposition
mqtt-broker	Mosquitto 2.1 (digest épinglé)	Broker MQTT : TLS 8883 pour les nœuds, 1883 interne	mosquitto (1883)	192.168.10.1:18883 → 8883
history-db	CouchDB 3.5 (digest épinglé)	Base documentaire : <code>telemetry</code> , <code>events</code>	couchdb (5984)	- (réseaux internes)
history-db-init	CouchDB (curl)	Job au démarrage : bases, comptes, droits, vues	couchdb (5984)	-
backend	Go 1.26, binaire statique	Ingestion MQTT, API REST, WebSocket	sentinel (10001)	-
dashboard	React 19 + Vite, nginx non privilégié	Interface, proxy <code>/api</code> , <code>/ws</code> , <code>/vision/</code>	nginx (101)	127.0.0.1:10443
ia-prediction	Python 3.12, scikit-learn 1.9	Isolation Forest en temps réel	ia (10001)	-
ia-vision	Python 3.12, PyTorch CPU, YOLOv8n	Détection d'intrusion, flux MJPEG	vision (10001)	- (webcam <code>/dev/video0</code>)

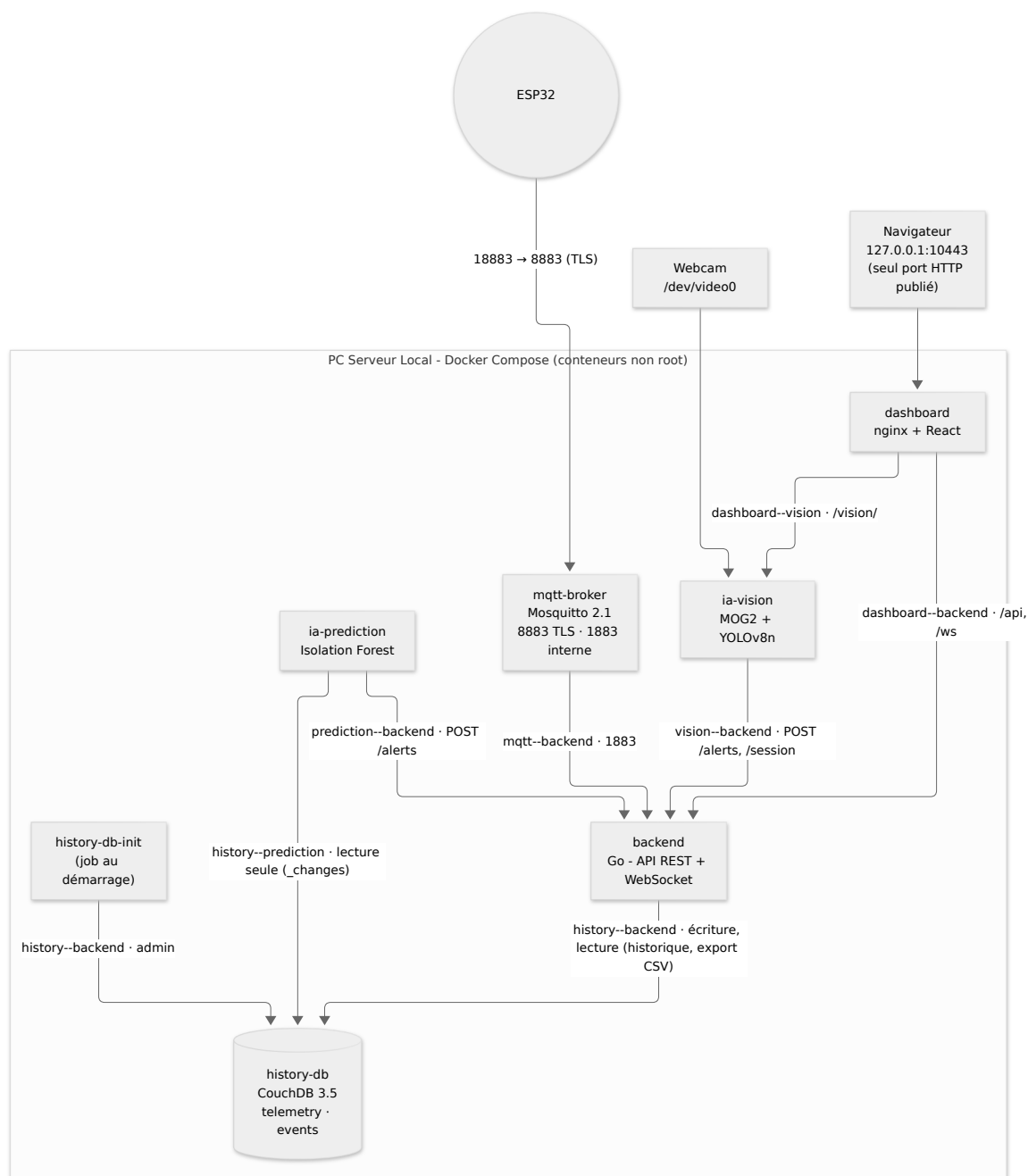


Figure 3 - Conteneurs, réseaux Docker et sens des échanges. Chaque réseau relie une paire de services.

5.2 Données

- **Bases** : `telemetry` (mesures) et `events` (mouvements, statuts, commandes, alertes, annotations), créées avec un seul shard (`q=1`) pour que le flux des changements reste ordonné.
- **Identifiants** : `<device_id>:<heure de réception en ms sur 13 chiffres>` : uniques et triés par le temps, ce qui rend les requêtes par plage de dates efficaces.
- **Écriture** : par lots (`_bulk_docs`) toutes les 2 s ; si CouchDB est indisponible, les documents attendent en mémoire (jusqu'à 20 000 par base) et sont réessayés.
- **Vues** : moyennes, minimums et maximums par minute et par heure (`_stats`) pour les graphiques longue durée.
- **Droits** : utilisateur `backend` (rôle `writer`), utilisateur `ia` (rôle `reader`, écriture refusée par `validate_doc_update`), design docs réservés à l'administrateur.

5.3 Construction et déploiement

- Les images applicatives sont construites depuis la branche `main` de chaque dépôt (contexte Git distant) : rien n'est copié à la main.
- Les images tierces sont **épinglées par empreinte** (`@sha256`) : une mise à jour en amont ne change rien sans décision explicite.
- Secrets dans `infra/.env` (ignoré par git) ; `scripts/gen-certs.sh` crée la CA et le certificat du broker, `scripts/mqtt-passwd.sh` les comptes MQTT.
- Construction avec Internet (`docker compose build`), puis fonctionnement hors ligne sur le hotspot (`docker compose up -d`) : le modèle YOLO est inclus dans l'image.

```
cp .env.example .env           # mots de passe et jetons
./scripts/gen-certs.sh 192.168.10.1 # CA locale + certificat du broker
./scripts/mqtt-passwd.sh        # comptes MQTT (backend, nœuds)
docker compose build            # avec Internet
docker compose up -d           # puis hors ligne, sur le hotspot
```

5.4 Maintien en condition opérationnelle

- **Redémarrage automatique** (`restart: unless-stopped`) et **contrôles de santé** (backend, dashboard).
- **Logs** : Mosquitto écrit sur la sortie standard, rotation Docker 3×10 Mo ; pas de croissance illimitée des journaux MQTT.
- **Supervision** : `GET /api/v1/health` (état MQTT et CouchDB, file d'écriture, messages rejetés), affiché dans le dashboard.
- **Ressources** : suivi par `docker stats` ; la pile complète utilise moins de 800 Mio de RAM ; le CPU est surtout pris par la vision (section 10).

6. Séquences des échanges réseau

6.1 Connexion du nœud et télémétrie

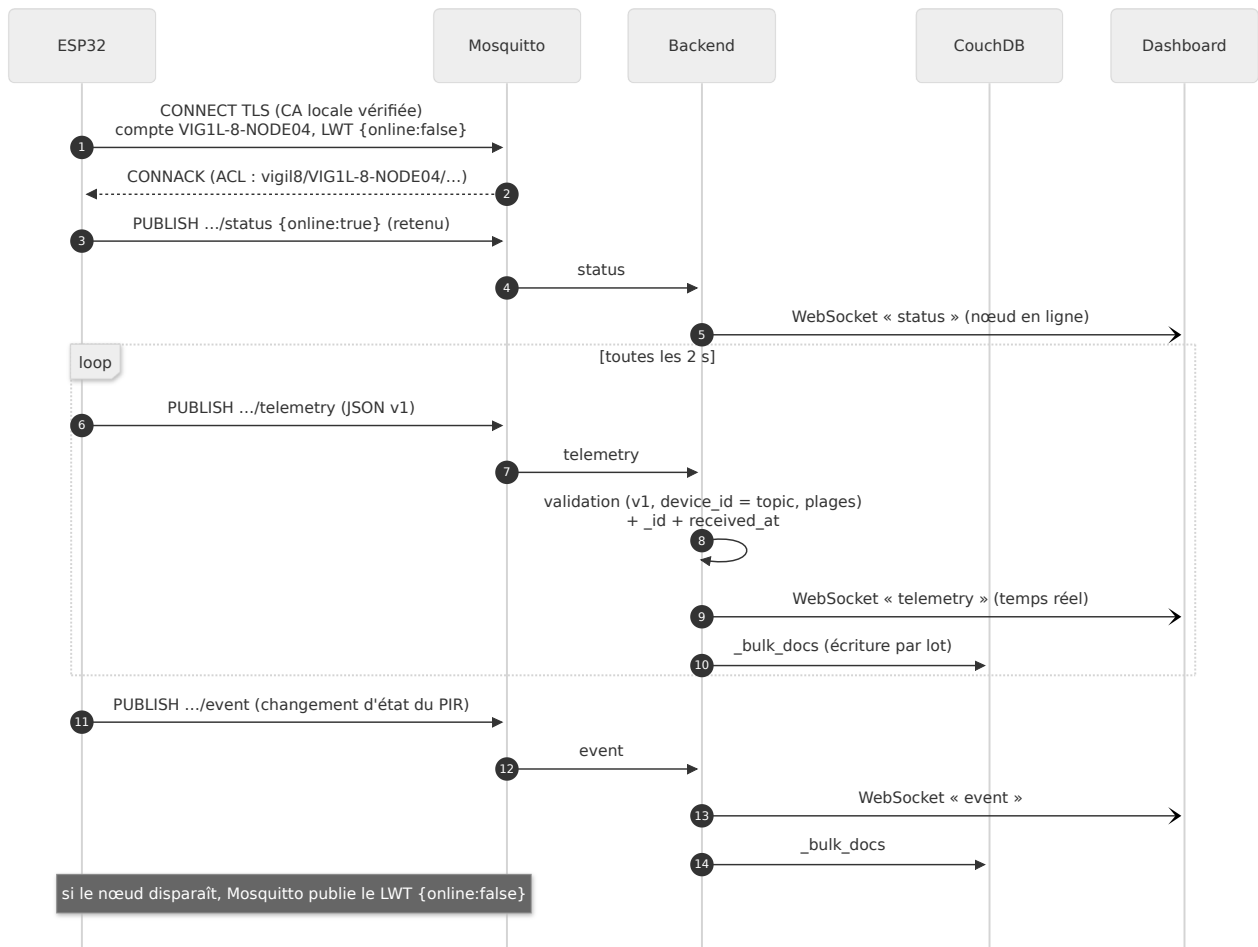


Figure 4 - Connexion TLS du nœud, télémétrie toutes les 2 s, événement PIR, déconnexion (LWT).

Le backend est le seul abonné à tous les nœuds. Il valide chaque message, ajoute l'heure de réception, le pousse immédiatement au dashboard par WebSocket et l'écrit en base par lots. Un message dont le `device_id` ne correspond pas au topic est rejeté et compté, même si l'ACL du broker l'a laissé passer.

6.2 Maintenance prédictive : alertes à deux niveaux

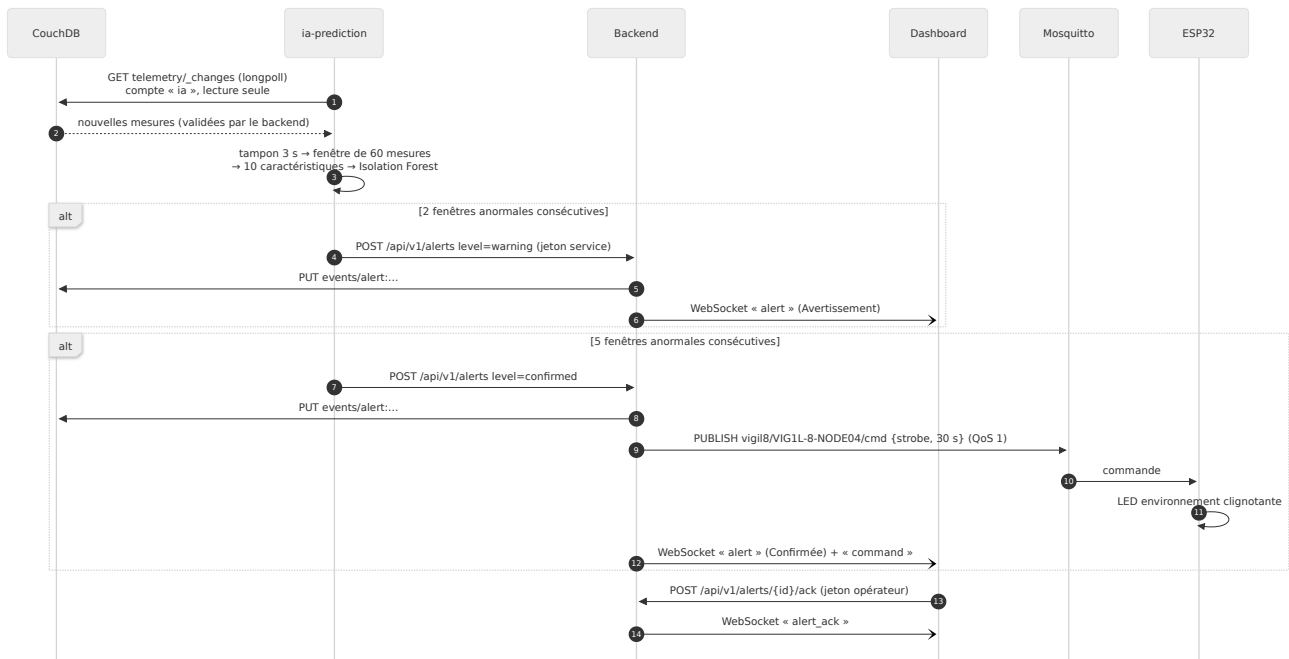


Figure 5 - De la mesure en base à la LED du nœud : avertissement, confirmation, acquittement.

6.3 Vision : intrusion et flux vidéo

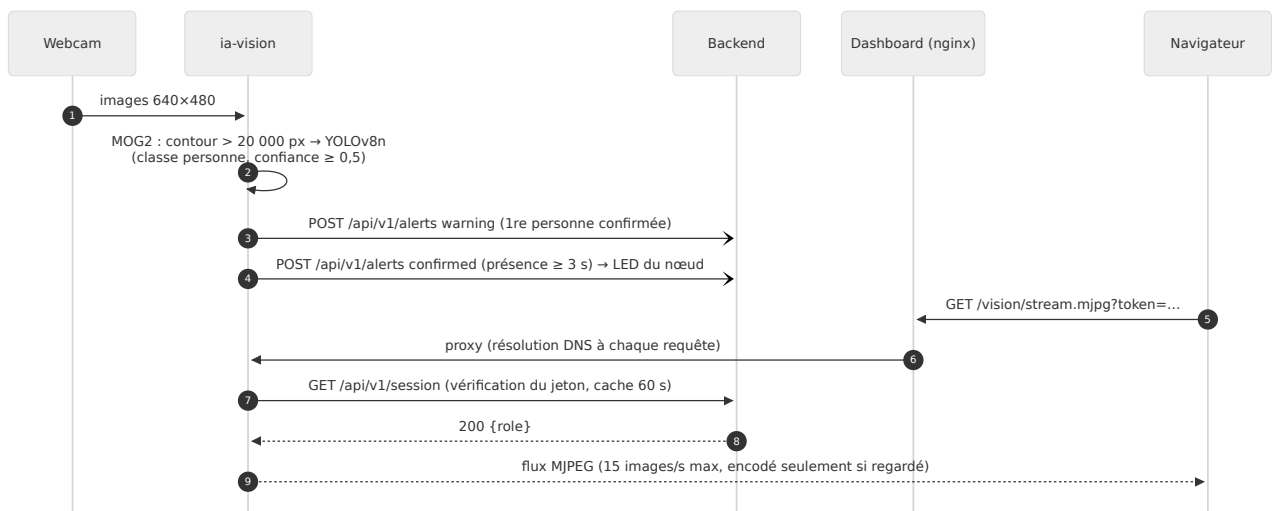


Figure 6 - Détection d'intrusion et affichage du flux, protégé par la clé d'accès.

7. Backend, API et dashboard

7.1 API

Toutes les routes, sauf `/api/v1/health`, exigent un jeton `Authorization: Bearer`. Référence complète : `backend/README.md`.

Route	Rôle	Usage
<code>GET /api/v1/devices</code>	service	état temps réel des nœuds
<code>GET /api/v1/devices/{id}/telemetry</code>	service	historique brut (≤ 6 h) ou agrégé par minute / heure
<code>GET /api/v1/devices/{id}/export.csv</code>	service	CSV d'entraînement de l'Isolation Forest
<code>POST /api/v1/alerts</code>	service	alerte de l'IA (<code>warning</code> / <code>confirmed</code>) - point d'entrée obligatoire du sujet
<code>POST /api/v1/alerts/{id}/ack</code>	opérateur	acquiescement
<code>POST /api/v1/annotations</code>	opérateur	période de test (exclue de l'entraînement)
<code>POST /api/v1/devices/{id}/command</code>	opérateur	LED environnement du nœud
<code>GET /ws</code>	service	WebSocket : instantané puis chaque message

7.2 Intégrité des données

- **Validation** : version du format, cohérence `device_id` / topic, plages physiques du DHT22, cohérence `dht` / valeurs.
- **Stockage fidèle** : le message est conservé tel quel ; le backend ajoute seulement `_id` et `received_at`.
- **Requêtes** : corps limités à 16 Ko, champs JSON inconnus refusés, identifiants validés par expressions régulières.
- **Traçabilité** : commandes, alertes et acquiescements sont enregistrés comme événements, avec leur source.

7.3 Dashboard

- Télémétrie en temps réel et historique (15 min à 24 h), capteurs en panne signalés (`ERR`), coupures visibles sur les courbes.
- Alertes à deux niveaux avec acquiescement, notification et son ; journal des événements.
- Commande de la LED, périodes de test (annotations), export CSV pour l'entraînement.
- Vision : état compact et flux vidéo affiché à la demande.
- Connexion par clé d'accès (jeton API) gardée en `sessionStorage` ; mode lecture seule avec le jeton service.
- Charte reprise de la présentation du projet : **état global de la table** toujours visible dans le rail latéral (nominal, vigilance, alerte, dégradé), bandeau épinglé tant qu'une alerte n'est pas acquiescée, panneaux numérotés par priorité.

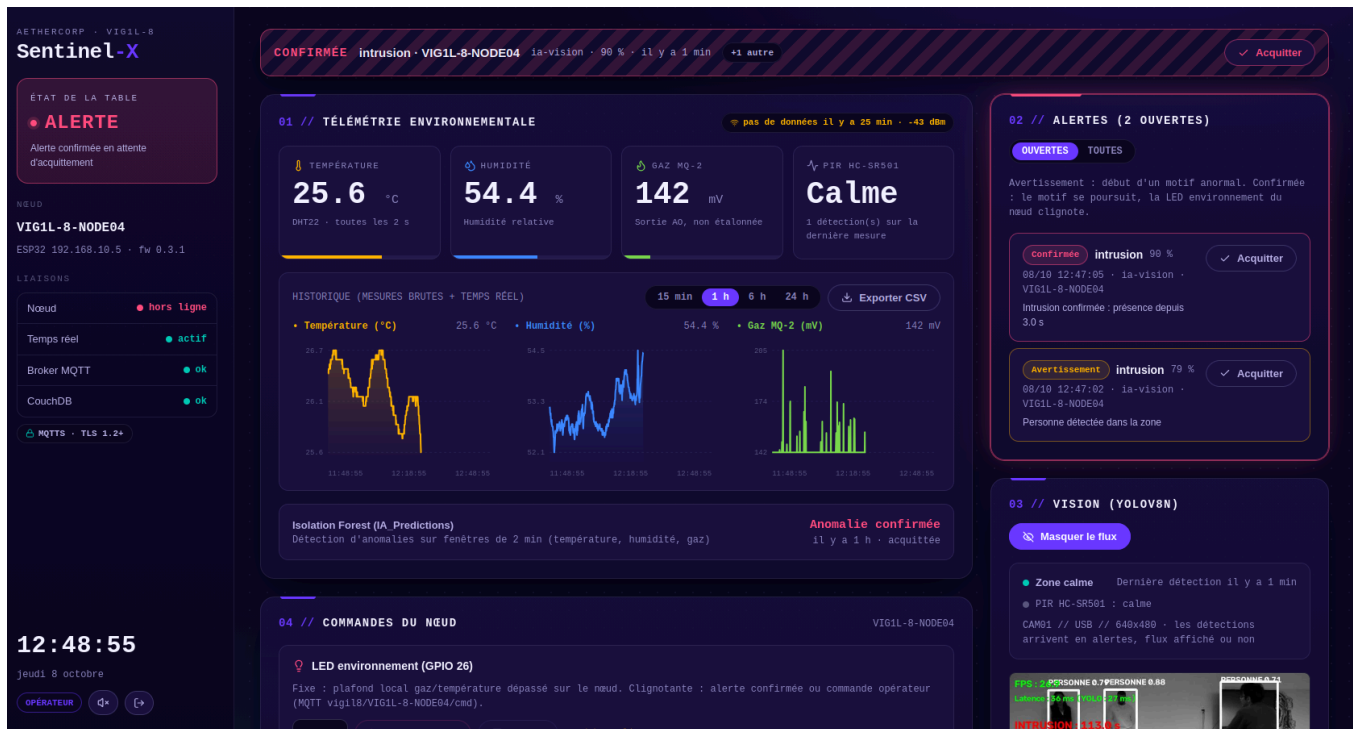


Figure 7 - Dashboard en fonctionnement : intrusion confirmée par la vision (bandeau et état « ALERTE »), alertes à acquiescer, historique d'une heure et flux de la webcam annoté.

8. Matrice de sécurité

8.1 Chiffrement TLS

Élément	Choix
Autorité	CA locale « Sentinel-X Local CA » (RSA 2048, 10 ans) ; clé privée jamais montée dans un conteneur
Certificat du broker	RSA 2048, SHA-256, 825 jours ; SAN : IP:192.168.10.1 , DNS:192.168.10.1 , 127.0.0.1 , mqtt-broker , localhost
Protocole	TLS 1.2 minimum sur le port 8883 ; le port 1883 sans TLS n'est joignable que sur les réseaux Docker internes
Vérification côté nœud	L'ESP32 embarque le certificat de la CA et vérifie celui du broker : un faux broker est refusé
Particularité	mbedtls 2.x de l'ESP32 ne compare que les noms DNS : l'IP est donc aussi déclarée comme nom DNS dans le certificat

8.2 Menaces et contrôles

Menace	Contrôle	État
Écoute / homme du milieu sur le Wi-Fi	MQTT sur TLS, certificat vérifié par le nœud ; Wi-Fi WPA2	FAIT
Connexion anonyme au broker	allow_anonymous false , mots de passe hachés	FAIT
Usurpation d'un nœud	Un compte par nœud ; ACL vigil8/%u/... ; contrôle device_id = topic dans le backend	FAIT
Injection de commandes vers un nœud	Seul le compte backend peut publier sur .../cmd ; côté API, jeton opérateur requis	FAIT
Accès à la base	Authentification obligatoire, port non publié sur l'hôte (réseaux Docker du backend et de l'IA uniquement), rôles écrivain / lecteur, validation d'écriture	FAIT
Accès à l'API	Jetons opérateur / service (comparaison en temps constant), aucun secret par défaut	FAIT
Déni de service applicatif	20 requêtes/s par IP, corps 16 Ko, délais HTTP, messages MQTT 4 Ko, 4 spectateurs vidéo max	FAIT
Accès au dashboard	Écoute sur 127.0.0.1 ; CSP stricte, X-Frame-Options , nosniff	FAIT
Flux de la caméra	Clé d'accès vérifiée auprès du backend ; clé absente des journaux nginx	FAIT
Fuite de secrets	.env , mots de passe MQTT, certificats, secrets.h ignorés par git ; ancien mot de passe Wi-Fi retiré du code	FAIT
Évasion de conteneur	Tous les services non root ; aucun conteneur privilégié ; seule la webcam est partagée	FAIT

Images compromises en amont	Images tierces épinglées par empreinte	FAIT
Attaques automatisées sur les ports par défaut	Ports non standard (MQTTS 18883, dashboard 10443) : hors de portée des scans et robots qui ne visent que les ports courants ; complément de TLS et de l'authentification	FAIT
Port exposé sur d'autres réseaux	MQTTS publié sur l'adresse du hotspot (192.168.10.1) uniquement : injoignable depuis le Wi-Fi de l'école ou l'Ethernet	FAIT
Accès distant à l'hôte	Aucun serveur SSH : le portable est administré sur place (section 8.3)	FAIT

8.3 Sécurité de l'hôte : périmètre retenu

Le sujet prévoit un durcissement du système hôte (pare-feu, SSH par clés, privilèges du démon Docker), pensé pour un serveur dédié administré à distance, comme le Raspberry Pi de l'option A. Avec l'option B, le serveur est le portable de l'opérateur, utilisé sur place. Nous avons donc concentré la sécurité sur ce qui est réellement exposé : le réseau de table et le seul port publié.

Mesure du sujet	Choix pour l'option B et justification
Fermeture des ports non requis (UFW / iptables)	La surface est réduite à la source plutôt que filtrée après coup. Un seul port est joignable depuis le réseau : MQTTS 18883, publié uniquement sur l'adresse du hotspot (192.168.10.1), protégé par TLS, un compte par nœud et des ACL. Le dashboard n'écoute que sur 127.0.0.1, la base et l'API ne sont pas publiées. Un pare-feu UFW n'apporterait rien de plus pour la pile : Docker contourne UFW pour les ports qu'il publie, c'est la liaison à l'adresse du hotspot qui fait foi. Vérification : <code>ss -ltnp</code> ne montre que 192.168.10.1:18883 et 127.0.0.1:10443 pour la pile.
SSH par clés uniquement	Non pertinent : le portable est administré directement par son utilisateur, aucun serveur SSH n'y est lancé. Sans service SSH, il n'y a ni mot de passe à deviner ni clé à protéger. Pour un serveur dédié (option A), l'accès se ferait par clés uniquement.
Isolation des privilèges du démon Docker	Traitée au niveau des conteneurs : tous les services tournent avec un utilisateur non root, aucun n'est privilégié, seule la webcam est partagée, chaque réseau Docker ne relie qu'une paire de services. Retirer l'utilisateur du groupe <code>docker</code> ou activer le remapping des UID protège contre un attaquant déjà connecté à la session du portable, ce qui sort du modèle de menace retenu (attaques par le réseau de table).
Réseau de table	Hotspot dédié du portable (WPA2), sous-réseau isolé 192.168.10.0/24 : les autres groupes ne partagent pas ce réseau.

9. Documentation de l'IA

9.1 Vision : détection d'intrusion

Un script Python (`IA_Vision`) analyse la webcam USB du serveur. Pour respecter la contrainte de moins de 100 ms par image, le modèle de détection n'est pas lancé sur chaque image : une **soustraction de fond** repère d'abord les mouvements, et seuls les grands mouvements déclenchent **YOLOv8n**.

Étape	Paramètres
Prétraitement	Image redimensionnée en 640×480, niveaux de gris, flou gaussien 21×21
Soustraction de fond (MOG2)	Historique 500, seuil 80, ombres détectées ; 60 premières images d'apprentissage du décor
Filtre de lumière	Changement sur plus de 40 % de l'image ignoré (allumage d'une lampe, nuage)
Déclenchement	Contour de plus de 20 000 px → YOLOv8n
Confirmation	YOLOv8n, classe « personne » uniquement, confiance $\geq 0,5$, entrée 320 px
Persistance	L'intrusion se termine après 2 s sans personne détectée
Alertes	<code>warning</code> à la première personne confirmée ; <code>confirmed</code> après 3 s de présence (LED du nœud)

Les alertes sont envoyées dans un thread séparé : la boucle vidéo n'attend jamais le réseau. Le flux MJPEG des images annotées n'est encodé que si quelqu'un le regarde.

Mesure	Tests (CPU, conteneur)
Images par seconde	24,5
Latence par image	37 ms (objectif < 100 ms)
Inférence YOLOv8n seule	38 ms

9.2 Maintenance prédictive : Isolation Forest

Pourquoi l'Isolation Forest

Le sujet interdit les seuils statiques et demande de détecter des anomalies « cinétiques », par exemple une hausse lente de la température associée à une micro-déviations du gaz, avant tout seuil critique. Nous n'avons **aucun exemple réel d'anomalie étiqueté** : un modèle supervisé (Random Forest) ne pourrait être entraîné que sur des données inventées. L'Isolation Forest est **non supervisé** : il apprend ce qui est normal à partir des vraies mesures et isole ce qui s'en écarte.

Données et caractéristiques

- **Mesures** : température, humidité, gaz, toutes les 2 s.
- **Fenêtres** : 60 mesures (2 minutes), calculées **par segment continu** (jamais à cheval sur une coupure).
- **10 caractéristiques par fenêtre** : moyenne (×3), écart-type (×3), pente par minute (×3, régression linéaire), corrélation température / gaz.
- **Modèle** : normalisation (`StandardScaler`) puis Isolation Forest (200 arbres, contamination 0,5 %).

Préparation des données d'entraînement

Pour que le modèle n'apprenne que du « normal », les mesures suivantes sont retirées :

- DHT22 en erreur ou valeurs manquantes ;
- préchauffage du MQ-2 (3 minutes après chaque démarrage) ;
- périodes de test annotées depuis le dashboard (briquet, souffle chaud...) ;
- segments de moins de 60 mesures.

Les données viennent du bouton « Exporter CSV » du dashboard (l'API lit la base et sert le fichier, pour un entraînement sur un autre PC), ou directement de CouchDB depuis le conteneur (`docker compose run --rm ia-prediction python entraînement.py --source couchdb`) : les deux appliquent exactement les mêmes règles.

Détection en temps réel

- Lecture des nouvelles mesures dans CouchDB (flux `_changes` , compte lecture seule), tampon de 3 s pour les remettre dans l'ordre.
- Au démarrage, les 2 dernières minutes sont rechargées : la détection reprend immédiatement, sans relancer d'alerte déjà envoyée.
- **Avertissement** après 2 fenêtres anormales consécutives (un motif commence), **confirmation** après 5 (le motif se poursuit, la LED du nœud clignote). Un seul envoi par niveau et par épisode ; l'épisode se termine après 30 s normales.

Évaluation

Jeu	Résultat
Données simulées normales (10 séries de 20 min)	0,07 % de fenêtres en fausse alerte
Dérive lente simulée (température, gaz, humidité)	Détectée 10 fois sur 10, en 0,8 min en moyenne
Pic de gaz simulé	Détecté 10 fois sur 10, en 0,2 min en moyenne
Chaîne complète, nœud simulé (modèle entraîné sur 900 mesures)	Avertissement à gaz = 266 mV (+8 mV au-dessus de la normale), confirmation 3 mesures plus tard, LED commandée

Modèle utilisé en démonstration : réentraîné sur une plage d'une heure de mesures réelles prises dans la salle (`entraînement.py --source couchdb`), avec les mêmes règles de nettoyage et de découpage en segments continus.

10. Tests et validation

En plus des tests unitaires (backend Go, avec détection des accès concurrents), chaque évolution a été validée sur une **pile complète** : infrastructure, backend, dashboard et IA, avec un nœud ESP32 simulé qui publie au même format, en MQTTS, avec son compte.

Domaine	Vérification	Résultat
TLS	Connexion sans la CA du projet	refusée
Broker	Connexion anonyme, mauvais mot de passe	refusées
ACL	Un nœud publie au nom d'un autre ; un nœud lit les commandes d'un autre ; l'IA publie une commande	bloqués
Backend	device_id différent du topic	rejeté et compté
CouchDB	Requête anonyme ; écriture par ia ; suppression d'une vue par backend	401 / 403 / 403
API	Sans jeton ; jeton service sur une commande ; champ JSON inconnu	401 / 403 / 400
WebSocket	Mauvais jeton ; origine étrangère	401 / 403
Vision	Flux sans clé ; avec clé	401 / 200
Prédiction	Redémarrage pendant une anomalie	aucun doublon
Export	CSV exporté puis entraînement	900 / 900 mesures, modèle entraîné

10.1 Mesures sur le système réel

Mesure	Résultat
Latence réseau ESP32 ↔ PC Serveur Local (hotspot)	1 à 2 ms
Vision, sur le PC de test (CPU, conteneur)	37 ms par image, 24,5 images/s (section 9.1)
Entraînement de l'Isolation Forest	1 h de mesures réelles dans la salle

Consommation de la pile complète sur le PC Serveur Local, les deux IA comprises (docker stats , 15,3 Gio de RAM ; 100 % = un cœur de processeur) :

Conteneur	CPU	Mémoire	Processus
ia-vision (PyTorch, YOLOv8n)	803 %	471,1 Mio	47
ia-prediction (Isolation Forest)	0,00 %	181,9 Mio	31
history-db (CouchDB)	1,33 %	107,8 Mio	79
dashboard (nginx)	0,00 %	12,5 Mio	17
mqtt-broker (Mosquitto)	0,02 %	6,4 Mio	1
backend (Go)	0,00 %	5,9 Mio	9
Total	≈ 805 %	≈ 786 Mio (5 %)	184

La mémoire tient largement sur le portable (moins de 800 Mio pour toute la pile). Le processeur est presque entièrement pris par la vision : PyTorch répartit l'analyse de chaque image sur tous les cœurs disponibles (environ 8 ici), alors que les autres services restent sous 2 %. Lors d'un relevé précédent, sans la vision, la pile utilisait environ 2 % de CPU et 133 Mio de RAM.

11. Limites et améliorations

Limite	Conséquence	Amélioration
Pas de durcissement du système hôte (pare-feu, SSH, groupe <code>docker</code>)	Acceptable pour un portable utilisé sur place (option B), pas pour un serveur dédié	Pour l'option A : UFW, SSH par clés uniquement, utilisateur hors du groupe <code>docker</code> , remapping des UID
Accès distant au dashboard	Sans objet en option B : le dashboard est affiché directement sur le portable serveur	Avec un serveur dédié (Raspberry Pi, option A) : reverse proxy HTTPS devant le dashboard et l'API, accès d'administration par VPN uniquement
Wi-Fi WPA2-PSK partagé	Qui connaît le mot de passe rejoint le réseau (le MQTT reste protégé)	WPA3, ou isolation des clients sur le hotspot
Authentification du nœud par mot de passe	Mot de passe lisible dans la mémoire flash si l'ESP32 est volé	Certificat client par nœud (mTLS), chiffrement de la flash
Clé d'accès dans l'URL du WebSocket et du flux vidéo	Les navigateurs ne permettent pas d'en-tête ; la clé est exclue des journaux	Jeton court à usage unique
Dashboard en HTTP	Acceptable car limité à <code>127.0.0.1</code>	HTTPS avec la CA locale si l'accès distant devient nécessaire
MQ-2 branché sans pont diviseur	Sous gaz fort, AO peut dépasser 3,3 V : l'ADC sature vers 3,1 V et l'entrée de l'ESP32 n'est pas protégée	Pont diviseur 10 kΩ / 20 kΩ (facteur 1,5 dans le firmware), puis réentraînement de l'IA sur la nouvelle échelle
MQ-2 non étalonné	Valeurs relatives (mV), pas de ppm	Étalonnage en air propre (R0), puis conversion par courbe
Vision gourmande en CPU	PyTorch occupe tous les cœurs du portable (≈ 800 %)	Limiter les threads de PyTorch (<code>torch.set_num_threads</code>) ou le CPU du conteneur (<code>cpus</code> : dans Compose), en vérifiant que la latence reste sous 100 ms
Filtre de lumière de la vision	Une personne très proche (plus de 40 % de l'image) peut être ignorée	Ajuster le seuil, ou lancer YOLO périodiquement
Modèle IA entraîné sur peu de données	Variations jour / nuit pas encore apprises	Collecter plusieurs jours, réentraîner régulièrement
Dépôts publics	Choix lié au contexte scolaire (construction sans identifiants)	Dépôts privés et clé de déploiement en lecture seule

12. Annexes

A. Dépôts

Dépôt	Contenu
infra	Docker Compose, configuration Mosquitto et CouchDB, scripts, ce rapport (docs/gen/ , PDF dans docs/)
backend	Service Go : ingestion MQTT, API REST, WebSocket, export CSV
dashboard	Interface React, configuration nginx
firmware	Firmware ESP32 (PlatformIO), diagnostic matériel
IA_Vision	Détection d'intrusion, flux MJPEG, image Docker
IA_Predictions	Isolation Forest : génération de données, entraînement, export, détection en temps réel

B. Comptes et secrets

Compte	Où	Droits
VIGIL-8-NODE04	Mosquitto	publie sur ses propres topics, lit ses commandes
backend	Mosquitto	lit tous les nœuds, publie les commandes
admin	CouchDB	administration, job d'initialisation uniquement
backend	CouchDB	rôle writer
ia	CouchDB	rôle reader (lecture seule)
jeton opérateur	API	tout, y compris commandes et acquittements
jeton service	API	lecture et envoi d'alertes (IA)

Tous les secrets sont dans `infra/.env`, `infra/config/mosquitto/passwd` (haché), `infra/certs/` et `firmware/include/secrets.h`, ignorés par git.